recommend_friend_models

2026-04-13

```r
library(MASS)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.5.2
## v ggplot2   4.0.0     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## x dplyr::select() masks MASS::select()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(Matrix)
```

```
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```r
library(glmnet)
```

```
## Loaded glmnet 4.1-10
```

```r
library(here)
```

```
## Warning: package 'here' was built under R version 4.5.3
```

```
## here() starts at C:/Users/roddu/Downloads/glassdoor-company-reviews
```

```r
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```r
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
library(broom)
library(tidytext)
```

```
## Warning: package 'tidytext' was built under R version 4.5.3
```

```r
set.seed(123)

df <- read.csv(here("data/processed/reviews_with_sent_scores.csv"))
```

Create Aggregated Sentiment Features

```r
df <- df %>%
  mutate(
    sent_vader_avg = rowMeans(
      across(c(
        vader_summary_compound,
        vader_pros_compound,
        vader_cons_compound,
        vader_advice_compound
      )),
      na.rm = TRUE
    ),
```

```r
  sent_vader_sd = apply(
    select(., c(
      vader_summary_compound,
      vader_pros_compound,
      vader_cons_compound,
      vader_advice_compound
    )),
    1,
    sd,
    na.rm = TRUE
  ),

  sent_distilbert_avg = rowMeans(
    across(c(
      distilbert_summary_signed,
      distilbert_pros_signed,
      distilbert_cons_signed,
      distilbert_advice_signed
    )),
    na.rm = TRUE
  ),

  sent_distilbert_sd = apply(
    select(., c(
      distilbert_summary_signed,
      distilbert_pros_signed,
      distilbert_cons_signed,
      distilbert_advice_signed
    )),
    1,
    sd,
    na.rm = TRUE
  ),

  sent_roberta_avg = rowMeans(
    across(c(
      roberta_summary_signed,
      roberta_pros_signed,
      roberta_cons_signed,
      roberta_advice_signed
    )),
    na.rm = TRUE
  ),

  sent_roberta_sd = apply(
    select(., c(
      roberta_summary_signed,
      roberta_pros_signed,
      roberta_cons_signed,
      roberta_advice_signed
    )),
    1,
    sd,
```

```
      na.rm = TRUE
    )
  )
```

Define Base Features and Helper Functions

```
base_features <- c(
  "rating_overall",
  "rating_culture_values",
  "rating_work_life",
  "rating_compensation_benefits",
  "rating_senior_leadership",
  "rating_career_opportunities",
  "rating_diversity_inclusion",
  "flags_business_outlook",
  "flags_ceo_approval",
  "employee_length",
  "employee_job_quit",
  "sector",
  "covid_related_review",
  "career_missing"

)

make_model_data <- function(df, extra_cols = NULL) {
  df %>%
    dplyr::select(
      flags_recommend_frend,
      all_of(base_features),
      all_of(extra_cols)
    ) %>%
    mutate(
      employee_job_quit = as.integer(employee_job_quit),
      career_missing = as.integer(career_missing),
      flags_recommend_frend = factor(
        flags_recommend_frend,
        levels = c("NEGATIVE", "POSITIVE")
      ),
      sector = as.factor(sector),
      #flags_business_outlook = as.integer(as.factor(flags_business_outlook)),
      #flags_ceo_approval = as.integer(as.factor(flags_ceo_approval))
      flags_business_outlook = as.numeric(recode(flags_business_outlook,
                                "NEGATIVE" = -1,"NEUTRAL"  = 0, "POSITIVE" = 1)),
      flags_ceo_approval = as.factor(flags_ceo_approval)
    )
}

f1_score <- function(actual, predicted) {
  tp <- sum(actual == "POSITIVE" & predicted == "POSITIVE")
  fp <- sum(actual == "NEGATIVE" & predicted == "POSITIVE")
  fn <- sum(actual == "POSITIVE" & predicted == "NEGATIVE")
```

```r
  precision <- ifelse((tp + fp) == 0, 0, tp / (tp + fp))
  recall <- ifelse((tp + fn) == 0, 0, tp / (tp + fn))

  if (precision + recall == 0) return(0)

  2 * precision * recall / (precision + recall)
}

evaluate_model <- function(prob, actual, thresholds = seq(0.1, 1.0, by = 0.05)) {
  f1_results <- tibble(threshold = thresholds) %>%
    mutate(
      F1 = purrr::map_dbl(threshold, ~{
        pred_class <- ifelse(prob > .x, "POSITIVE", "NEGATIVE")
        f1_score(actual, pred_class)
      })
    )

  best_threshold <- f1_results %>%
    slice_max(F1, n = 1, with_ties = FALSE) %>%
    pull(threshold)

  pred_class <- factor(
    ifelse(prob > best_threshold, "POSITIVE", "NEGATIVE"),
    levels = c("NEGATIVE", "POSITIVE")
  )

  cm <- confusionMatrix(pred_class, actual)
  roc_obj <- roc(actual, prob, levels = c("NEGATIVE", "POSITIVE"), direction = "<")

  list(
    f1_results = f1_results,
    best_threshold = best_threshold,
    pred_class = pred_class,
    confusion_matrix = cm,
    roc = roc_obj,
    auc = as.numeric(auc(roc_obj))
  )
}

extract_glm_importance <- function(model, features_name, model_name, top_n = 10) {
  broom::tidy(model) %>%
    dplyr::filter(term != "(Intercept)") %>%
    dplyr::mutate(
      Features = features_name,
      Model = model_name,
      Importance = abs(estimate),
      Direction = ifelse(estimate >= 0, "Positive", "Negative")
    ) %>%
    dplyr::arrange(desc(Importance)) %>%
    dplyr::slice_head(n = top_n) %>%
    dplyr::select(Features, Model, term, estimate, Importance, Direction)
}
```

```r
extract_glmnet_importance <- function(cv_model, features_name, model_name, s = "lambda.min", top_n = 10)
  coef_df <- as.matrix(coef(cv_model, s = s)) %>%
    as.data.frame() %>%
    tibble::rownames_to_column("term")

  names(coef_df)[2] <- "estimate"

  coef_df %>%
    dplyr::filter(term != "(Intercept)", estimate != 0) %>%
    dplyr::mutate(
      Features = features_name,
      Model = model_name,
      Importance = abs(estimate),
      Direction = ifelse(estimate >= 0, "Positive", "Negative")
    ) %>%
    dplyr::arrange(desc(Importance)) %>%
    dplyr::slice_head(n = top_n) %>%
    dplyr::select(Features, Model, term, estimate, Importance, Direction)
}

extract_rf_importance <- function(rf_model_tuned, features_name, model_name, top_n = 10) {
  imp <- importance(rf_model_tuned$finalModel, type = 1)

  data.frame(
    term = rownames(imp),
    Importance = imp[, 1],
    stringsAsFactors = FALSE
  ) %>%
    dplyr::mutate(
      Features = features_name,
      Model = model_name,
      estimate = Importance,
      Direction = "RF Importance"
    ) %>%
    dplyr::arrange(desc(Importance)) %>%
    dplyr::slice_head(n = top_n) %>%
    dplyr::select(Features, Model, term, estimate, Importance, Direction)
}

clean_feature_names <- function(x) {
  x %>%
    stringr::str_replace_all("_", " ") %>%
    stringr::str_replace_all("rating career opportunities", "career opportunities") %>%
    stringr::str_replace_all("rating compensation benefits", "comp benefits") %>%
    stringr::str_replace_all("rating senior leadership", "senior leadership") %>%
    stringr::str_replace_all("rating culture values", "culture values") %>%
    stringr::str_replace_all("rating work life", "work life") %>%
    stringr::str_replace_all("rating diversity inclusion", "diversity inclusion") %>%
    stringr::str_replace_all("flags business outlook", "business outlook") %>%
    stringr::str_replace_all("flags ceo approval", "ceo approval") %>%
    stringr::str_replace_all("employee job quit", "job quit") %>%
    stringr::str_replace_all("sent vader avg", "vader avg") %>%
    stringr::str_replace_all("sent vader sd", "vader sd") %>%
```

```r
    stringr::str_replace_all("sent distilbert avg", "distilbert avg") %>%
    stringr::str_replace_all("sent distilbert sd", "distilbert sd") %>%
    stringr::str_replace_all("sent roberta avg", "roberta avg") %>%
    stringr::str_replace_all("sent roberta sd", "roberta sd")
}
```

Build Datasets

```r
datasets <- list(
  base_only = make_model_data(df),
  vader = make_model_data(df, c("sent_vader_avg", "sent_vader_sd")),
  distilbert = make_model_data(df, c("sent_distilbert_avg", "sent_distilbert_sd")),
  roberta = make_model_data(df, c("sent_roberta_avg", "sent_roberta_sd")),
  base_and_all_sent = make_model_data(df, c(
    "sent_vader_avg", "sent_vader_sd",
    "sent_distilbert_avg", "sent_distilbert_sd",
    "sent_roberta_avg", "sent_roberta_sd"
  ))
)

train_index <- createDataPartition(
  datasets$base_only$flags_recommend_frend,
  p = 0.8,
  list = FALSE
)
```

1. Fit Base-Only Logistic Models

```r
base_train <- datasets$base_only[train_index, ]
base_test  <- datasets$base_only[-train_index, ]

x_train_base <- model.matrix(flags_recommend_frend ~ ., data = base_train)[, -1]
x_test_base  <- model.matrix(flags_recommend_frend ~ ., data = base_test)[, -1]
y_train_base <- ifelse(base_train$flags_recommend_frend == "POSITIVE", 1, 0)

base_logit <- glm(
  flags_recommend_frend ~ .,
  data = base_train,
  family = binomial
)
base_logit_prob <- predict(base_logit, newdata = base_test, type = "response")
base_logit_eval <- evaluate_model(base_logit_prob, base_test$flags_recommend_frend)

base_lasso <- cv.glmnet(
  x_train_base,
  y_train_base,
  family = "binomial",
  alpha = 1
)
```

```r
base_lasso_prob <- as.numeric(
  predict(base_lasso, newx = x_test_base, s = "lambda.min", type = "response")
)
base_lasso_eval <- evaluate_model(base_lasso_prob, base_test$flags_recommend_frend)

base_ridge <- cv.glmnet(
  x_train_base,
  y_train_base,
  family = "binomial",
  alpha = 0
)
base_ridge_prob <- as.numeric(
  predict(base_ridge, newx = x_test_base, s = "lambda.min", type = "response")
)
base_ridge_eval <- evaluate_model(base_ridge_prob, base_test$flags_recommend_frend)

base_enet <- cv.glmnet(
  x_train_base,
  y_train_base,
  family = "binomial",
  alpha = 0.5
)
base_enet_prob <- as.numeric(
  predict(base_enet, newx = x_test_base, s = "lambda.min", type = "response")
)
base_enet_eval <- evaluate_model(base_enet_prob, base_test$flags_recommend_frend)
```

1.1 Summarize Base-Only Logistic Models

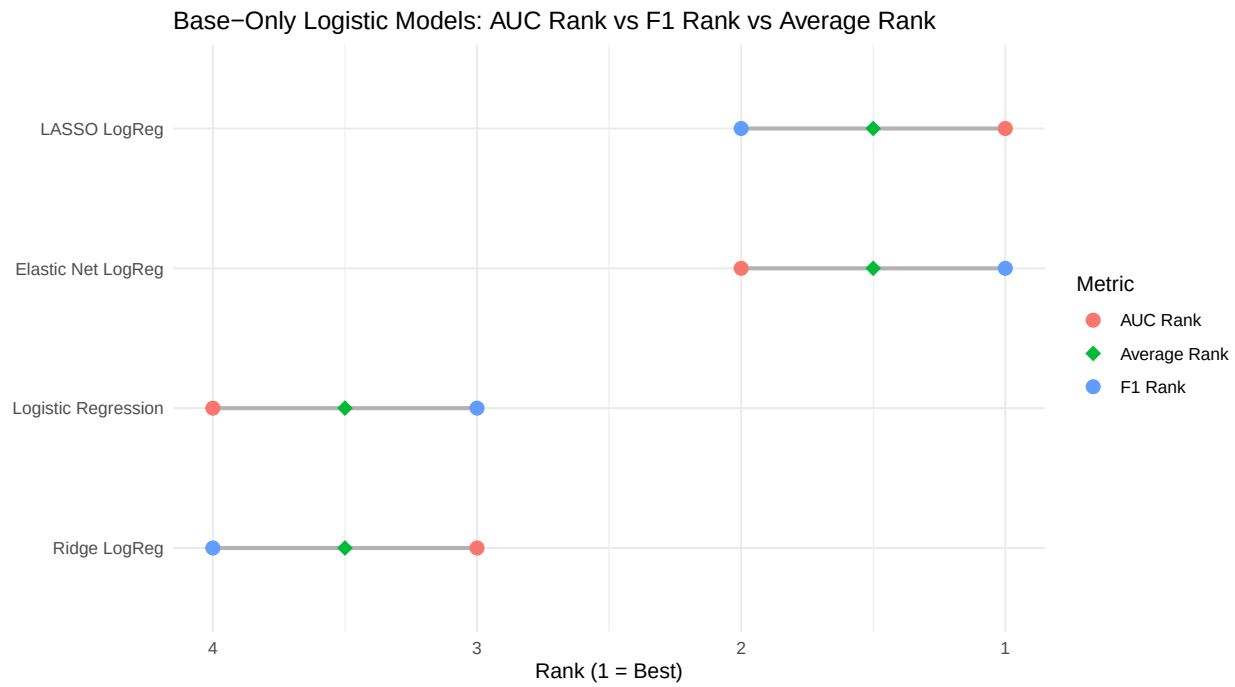
```r
base_logit_summary <- tibble(
  Features = "base_only",
  Model = c("Logistic Regression", "LASSO LogReg", "Ridge LogReg", "Elastic Net LogReg"),
  AUC = c(
    base_logit_eval$auc,
    base_lasso_eval$auc,
    base_ridge_eval$auc,
    base_enet_eval$auc
  ),
  F1 = c(
    max(base_logit_eval$f1_results$F1),
    max(base_lasso_eval$f1_results$F1),
    max(base_ridge_eval$f1_results$F1),
    max(base_enet_eval$f1_results$F1)
  ),
  Accuracy = c(
    as.numeric(base_logit_eval$confusion_matrix$overall["Accuracy"]),
    as.numeric(base_lasso_eval$confusion_matrix$overall["Accuracy"]),
    as.numeric(base_ridge_eval$confusion_matrix$overall["Accuracy"]),
    as.numeric(base_enet_eval$confusion_matrix$overall["Accuracy"])
  ),
  Best_Threshold = c(
```

```
    base_logit_eval$best_threshold,
    base_lasso_eval$best_threshold,
    base_ridge_eval$best_threshold,
    base_enet_eval$best_threshold
  )
)

base_logit_summary
```

```
## # A tibble: 4 x 6
##   Features  Model                 AUC    F1 Accuracy Best_Threshold
##   <chr>     <chr>               <dbl> <dbl>    <dbl>          <dbl>
## 1 base_only Logistic Regression 0.924 0.797    0.869           0.25
## 2 base_only LASSO LogReg        0.941 0.803    0.879           0.35
## 3 base_only Ridge LogReg        0.933 0.794    0.864           0.25
## 4 base_only Elastic Net LogReg  0.940 0.807    0.889           0.55
```

1.3 Plot Base-Only Logistic Models by AUC Rank, F1 Rank, and Average Rank

```
base_logit_rank_plot <- base_logit_summary %>%
  mutate(
    Model_Label = Model,
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
  arrange(Avg_Rank) %>%
  mutate(Model_Label = factor(Model_Label, levels = rev(Model_Label)))

ggplot(base_logit_rank_plot, aes(y = Model_Label)) +
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +
  geom_point(aes(x = AUC_Rank, color = "AUC Rank"), size = 3) +
  geom_point(aes(x = F1_Rank, color = "F1 Rank"), size = 3) +
  geom_point(aes(x = Avg_Rank, color = "Average Rank"), size = 3.5, shape = 18) +
  scale_x_reverse() +
  theme_minimal() +
  labs(
    title = "Base-Only Logistic Models: AUC Rank vs F1 Rank vs Average Rank",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  )
```



2. Fit Base-Only Random Forest

```
p_base <- ncol(base_train) - 1

rf_base_tune <- train(
  flags_recommend_frend ~ .,
  data = base_train,
  method = "rf",
  trControl = trainControl(method = "cv", number = 10),
  tuneGrid = expand.grid(mtry = 1:p_base),
  ntree = 500,
  importance = TRUE
)

rf_base_best <- rf_base_tune$bestTune

rf_base <- train(
  flags_recommend_frend ~ .,
  data = base_train,
  method = "rf",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = twoClassSummary
  ),
  tuneGrid = rf_base_best,
  ntree = 500,
  metric = "ROC",
  importance = TRUE
)
```

```r
rf_base_prob <- predict(rf_base, newdata = base_test, type = "prob")[, "POSITIVE"]
rf_base_eval <- evaluate_model(rf_base_prob, base_test$flags_recommend_frend)


rf_base_summary <- tibble(
  Features = "base_only",
  Model = "Random Forest",
  AUC = rf_base_eval$auc,
  F1 = max(rf_base_eval$f1_results$F1),
  Accuracy = as.numeric(rf_base_eval$confusion_matrix$overall["Accuracy"]),
  Best_Threshold = rf_base_eval$best_threshold
)

rf_base_summary
```

```
## # A tibble: 1 x 6
##   Features  Model           AUC    F1 Accuracy Best_Threshold
##   <chr>     <chr>         <dbl> <dbl>    <dbl>          <dbl>
## 1 base_only Random Forest 0.934 0.817    0.889            0.4
```

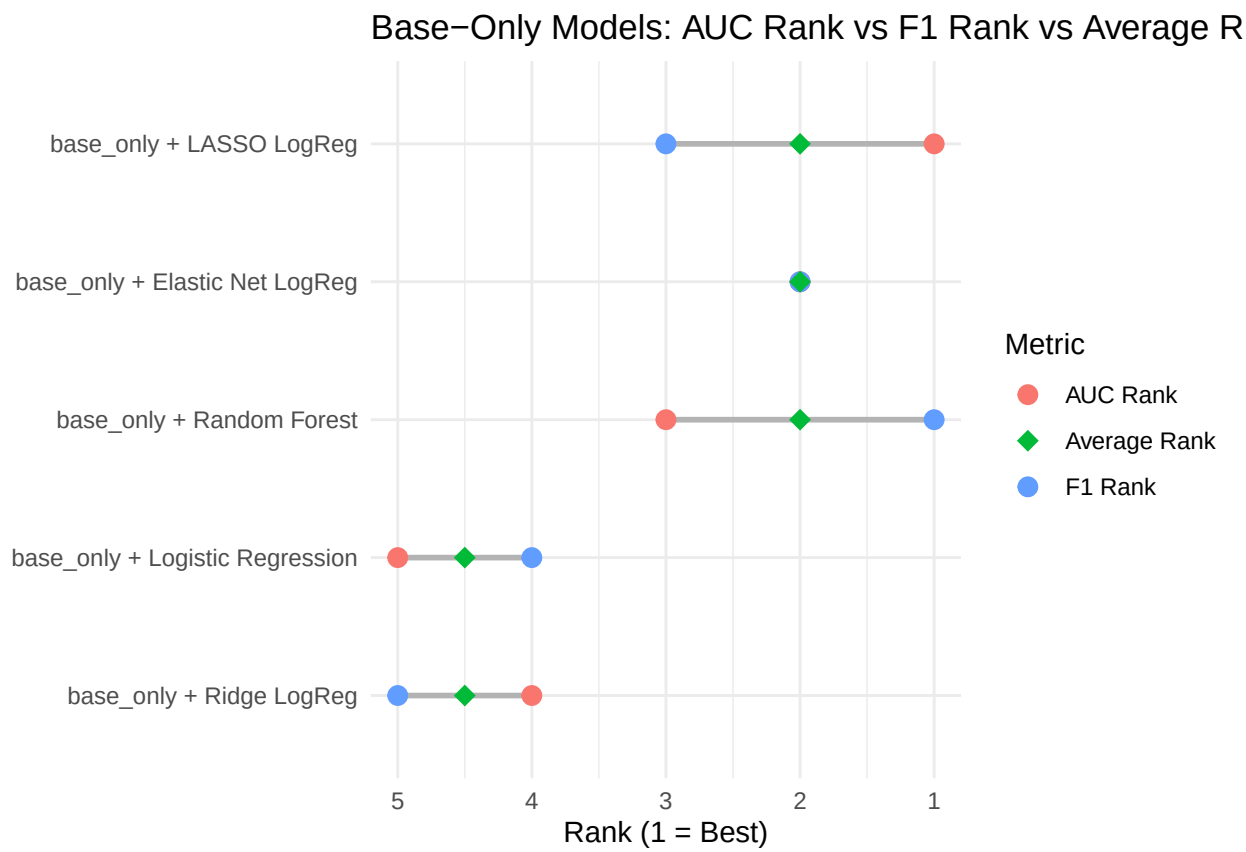
3 Compare Base-Only Models

```r
base_only_summary <- bind_rows(
  base_logit_summary %>% dplyr::select(Features, Model, AUC, F1, Accuracy),
  rf_base_summary %>% dplyr::select(Features, Model, AUC, F1, Accuracy)
) %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
  arrange(Avg_Rank)

base_only_summary
```

```
## # A tibble: 5 x 9
##   Features  Model       AUC    F1 Accuracy Model_Label AUC_Rank F1_Rank Avg_Rank
##   <chr>     <chr>     <dbl> <dbl>    <dbl> <chr>          <int>   <int>    <dbl>
## 1 base_only LASSO Lo~ 0.941 0.803    0.879 base_only ~        1       3      2
## 2 base_only Elastic ~ 0.940 0.807    0.889 base_only ~        2       2      2
## 3 base_only Random F~ 0.934 0.817    0.889 base_only ~        3       1      2
## 4 base_only Logistic~ 0.924 0.797    0.869 base_only ~        5       4      4.5
## 5 base_only Ridge Lo~ 0.933 0.794    0.864 base_only ~        4       5      4.5
```

3.1 Plot Base-Only Model Comparison

```r
base_only_summary_plot <- base_only_summary %>%
  arrange(Avg_Rank) %>%
  mutate(Model_Label = factor(Model_Label, levels = rev(Model_Label)))

ggplot(base_only_summary_plot, aes(y = Model_Label)) +
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +
  geom_point(aes(x = AUC_Rank, color = "AUC Rank"), size = 3) +
  geom_point(aes(x = F1_Rank, color = "F1 Rank"), size = 3) +
  geom_point(aes(x = Avg_Rank, color = "Average Rank"), size = 3.5, shape = 18) +
  scale_x_reverse() +
  theme_minimal() +
  labs(
    title = "Base-Only Models: AUC Rank vs F1 Rank vs Average Rank",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  )
```



Interpretation: Across base-only models, performance is driven primarily by structured rating features. Logistic regression and its regularized variants perform similarly, indicating that multicollinearity is present but not severe enough to drastically impact performance.

Random Forest captures nonlinear relationships and interactions, but does not consistently outperform linear models, suggesting the signal in the base features is largely linear.

Overall, base features alone provide a strong baseline, with limited gains from model complexity.

4 Extract Top 10 Base-Only Features for Each Model

```
base_logit_importance <- extract_glm_importance(
  base_logit,
  "base_only",
  "Logistic Regression",
  top_n = 10
)

base_lasso_importance <- extract_glmnet_importance(
  base_lasso,
  "base_only",
  "LASSO LogReg",
  top_n = 10
)

base_ridge_importance <- extract_glmnet_importance(
  base_ridge,
  "base_only",
  "Ridge LogReg",
  top_n = 10
)

base_enet_importance <- extract_glmnet_importance(
  base_enet,
  "base_only",
  "Elastic Net LogReg",
  top_n = 10
)

base_rf_importance <- extract_rf_importance(
  rf_base,
  "base_only",
  "Random Forest",
  top_n = 10
)

base_importance_all <- bind_rows(
  base_logit_importance,
  base_lasso_importance,
  base_ridge_importance,
  base_enet_importance,
  base_rf_importance
) %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    term_clean = clean_feature_names(term)
```

```r
  )

base_importance_all
```

```
## # A tibble: 50 x 8
##    Features  Model    term  estimate Importance Direction Model_Label term_clean
##    <chr>     <chr>    <chr>    <dbl>      <dbl> <chr>     <chr>       <chr>
##  1 base_only Logisti~ sect~    11.6       11.6  Positive  base_only ~ sectorLeg~
##  2 base_only Logisti~ sect~    11.5       11.5  Positive  base_only ~ sectorEne~
##  3 base_only Logisti~ flag~    11.4       11.4  Positive  base_only ~ ceo appro~
##  4 base_only Logisti~ flag~    11.2       11.2  Positive  base_only ~ ceo appro~
##  5 base_only Logisti~ sect~    11.2       11.2  Positive  base_only ~ sectorRea~
##  6 base_only Logisti~ flag~    10.2       10.2  Positive  base_only ~ ceo appro~
##  7 base_only Logisti~ sect~    10.1       10.1  Positive  base_only ~ sectorCon~
##  8 base_only Logisti~ sect~     9.79       9.79 Positive  base_only ~ sectorEdu~
##  9 base_only Logisti~ sect~     9.61       9.61 Positive  base_only ~ sectorTra~
## 10 base_only Logisti~ sect~     9.54       9.54 Positive  base_only ~ sectorMed~
## # i 40 more rows
```

4.1 Plot Top 10 Features for Base-Only Logistic-Family Models

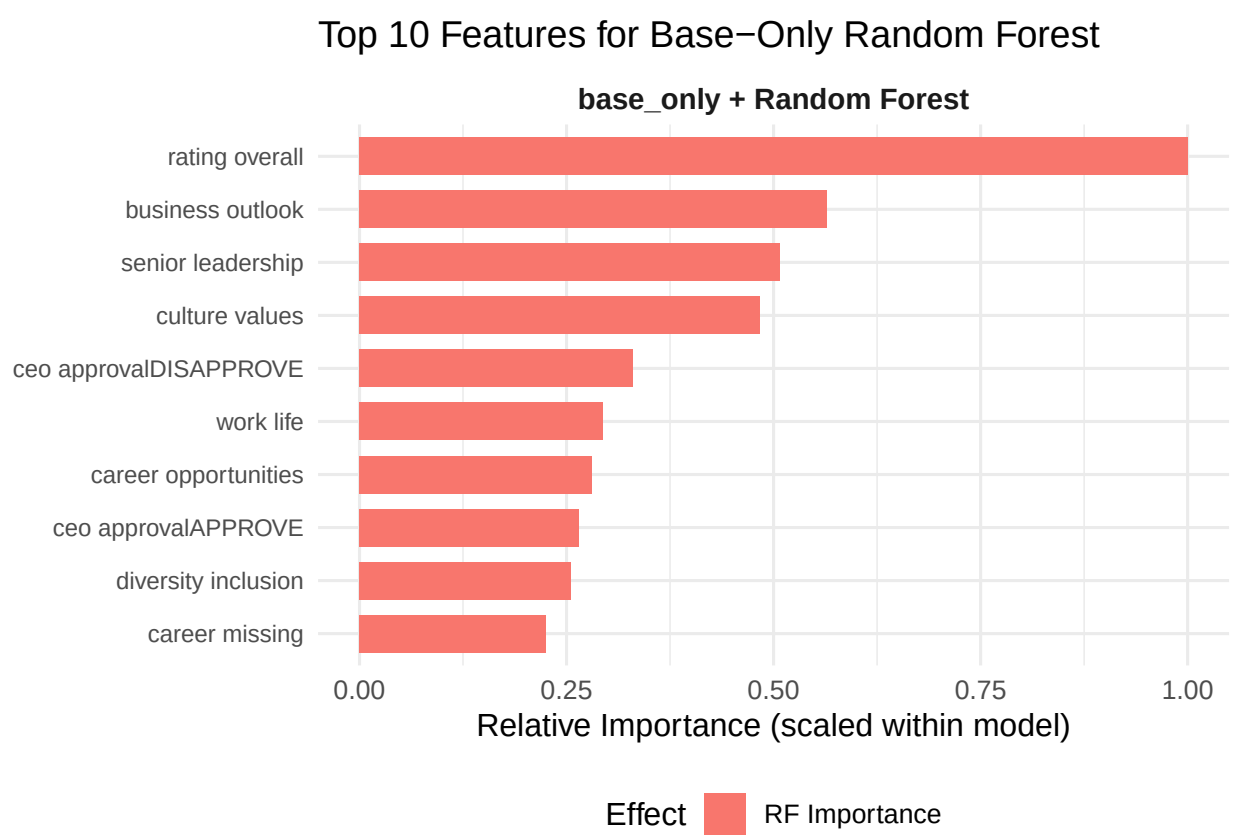
```r
base_logit_features_df <- base_importance_all %>%
  filter(Model != "Random Forest") %>%
  group_by(Model_Label) %>%
  slice_max(order_by = Importance, n = 10, with_ties = FALSE) %>%
  mutate(
    Importance_scaled = Importance / max(Importance)
  ) %>%
  ungroup()

base_logit_features_plot <- base_logit_features_df %>%
  mutate(
    term_plot = tidytext::reorder_within(term_clean, Importance_scaled, Model_Label),
    Direction = factor(Direction, levels = c("Positive", "Negative"))
  )

ggplot(base_logit_features_plot, aes(x = term_plot, y = Importance_scaled, fill = Direction)) +
  geom_col(width = 0.7) +
  coord_flip() +
  tidytext::scale_x_reordered() +
  facet_wrap(~ Model_Label, ncol = 2, scales = "free_y") +
  theme_minimal(base_size = 12) +
  labs(
    title = "Top 10 Features for Base-Only Logistic-Family Models",
    x = NULL,
    y = "Relative Importance (scaled within model)",
    fill = "Effect"
  ) +
  theme(
    axis.text.y = element_text(size = 9),
```

```
    strip.text = element_text(size = 11, face = "bold"),
    legend.position = "bottom",
    panel.spacing = unit(1.2, "lines")
  )
```

4.2 Plot Top 10 Features for Base-Only Random Forest Model

```
base_rf_features_df <- base_importance_all %>%
  filter(Model == "Random Forest") %>%
  group_by(Model_Label) %>%
  slice_max(order_by = Importance, n = 10, with_ties = FALSE) %>%
  mutate(
    Importance_scaled = Importance / max(Importance)
  ) %>%
  ungroup()

base_rf_features_plot <- base_rf_features_df %>%
  mutate(
```

```r
    term_plot = tidytext::reorder_within(term_clean, Importance_scaled, Model_Label),
    Direction = factor(Direction, levels = c("RF Importance"))
  )

ggplot(base_rf_features_plot, aes(x = term_plot, y = Importance_scaled, fill = Direction)) +
  geom_col(width = 0.7) +
  coord_flip() +
  tidytext::scale_x_reordered() +
  facet_wrap(~ Model_Label, ncol = 1, scales = "free_y") +
  theme_minimal(base_size = 12) +
  labs(
    title = "Top 10 Features for Base-Only Random Forest",
    x = NULL,
    y = "Relative Importance (scaled within model)",
    fill = "Effect"
  ) +
  theme(
    axis.text.y = element_text(size = 9),
    strip.text = element_text(size = 11, face = "bold"),
    legend.position = "bottom",
    panel.spacing = unit(1.2, "lines")
  )
```



Interpretation The most important predictors across base-only models are consistently rating_overall, flags_business_outlook, and career-related ratings. This suggests that employees' overall satisfaction and perception of company direction are the strongest drivers of recommendation behavior.

Regularized models (LASSO / Elastic Net) shrink less important variables, highlighting a smaller core set of influential predictors. Random Forest, in contrast, distributes importance more broadly, capturing additional secondary effects.

5. Fit Logistic Sentiment-Addons

Why Add Sentiment Features?

While structured features (ratings, flags, word counts) capture explicit signals in employee reviews, they may miss the underlying tone and nuance in textual feedback. Sentiment analysis helps quantify this tone by transforming free-text fields into numerical features that reflect positivity, negativity, and variability across different parts of a review.

We incorporated three sentiment models:

- VADER: A rule-based sentiment analyzer designed for short, informal text. It produces a compound score capturing overall sentiment intensity.
- DistilBERT (distilbert-base-uncased-finetuned-sst-2-english): A lightweight transformer model fine-tuned for sentiment classification, providing strong contextual understanding with lower computational cost.
- RoBERTa (cardiffnlp/twitter-roberta-base-sentiment-latest): A more powerful transformer model trained on large-scale data, capable of capturing subtle linguistic nuances and contextual sentiment shifts.

By adding these features, we aim to test whether textual tone provides additional predictive power beyond structured variables.

```r
logit_results <- list()
logit_importance <- list()

for (name in names(datasets)) {
  df_current <- datasets[[name]]

  train_df <- df_current[train_index, ]
  test_df  <- df_current[-train_index, ]

  x_train <- model.matrix(flags_recommend_frend ~ ., data = train_df)[, -1]
  x_test  <- model.matrix(flags_recommend_frend ~ ., data = test_df)[, -1]
  y_train <- ifelse(train_df$flags_recommend_frend == "POSITIVE", 1, 0)

  model_logit <- glm(
    flags_recommend_frend ~ .,
    data = train_df,
    family = binomial
  )
  prob_logit <- predict(model_logit, newdata = test_df, type = "response")
  eval_logit <- evaluate_model(prob_logit, test_df$flags_recommend_frend)

  cv_lasso <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 1)
  prob_lasso <- as.numeric(
    predict(cv_lasso, newx = x_test, s = "lambda.min", type = "response")
  )
```

```r
  eval_lasso <- evaluate_model(prob_lasso, test_df$flags_recommend_frend)

  cv_ridge <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 0)
  prob_ridge <- as.numeric(
    predict(cv_ridge, newx = x_test, s = "lambda.min", type = "response")
  )
  eval_ridge <- evaluate_model(prob_ridge, test_df$flags_recommend_frend)

  cv_enet <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 0.5)
  prob_enet <- as.numeric(
    predict(cv_enet, newx = x_test, s = "lambda.min", type = "response")
  )
  eval_enet <- evaluate_model(prob_enet, test_df$flags_recommend_frend)

  logit_results[[name]] <- list(
    logit_model = model_logit,
    logit_eval = eval_logit,
    lasso_model = cv_lasso,
    lasso_eval = eval_lasso,
    ridge_model = cv_ridge,
    ridge_eval = eval_ridge,
    enet_model = cv_enet,
    enet_eval = eval_enet
  )

  logit_importance[[paste0(name, "_logit")]] <- extract_glm_importance(
    model_logit, name, "Logistic Regression"
  )
  logit_importance[[paste0(name, "_lasso")]] <- extract_glmnet_importance(
    cv_lasso, name, "LASSO LogReg"
  )
  logit_importance[[paste0(name, "_ridge")]] <- extract_glmnet_importance(
    cv_ridge, name, "Ridge LogReg"
  )
  logit_importance[[paste0(name, "_enet")]] <- extract_glmnet_importance(
    cv_enet, name, "Elastic Net LogReg"
  )
}
```

5.1 Summarize Logistic Models with Sentiment Add-Ons

```r
model_comparison <- do.call(rbind, lapply(names(logit_results), function(name) {
  res <- logit_results[[name]]

  data.frame(
    Features = name,
    Model = c("Logistic Regression", "LASSO LogReg", "Ridge LogReg", "Elastic Net LogReg"),
    AUC = c(
      res$logit_eval$auc,
      res$lasso_eval$auc,
      res$ridge_eval$auc,
```

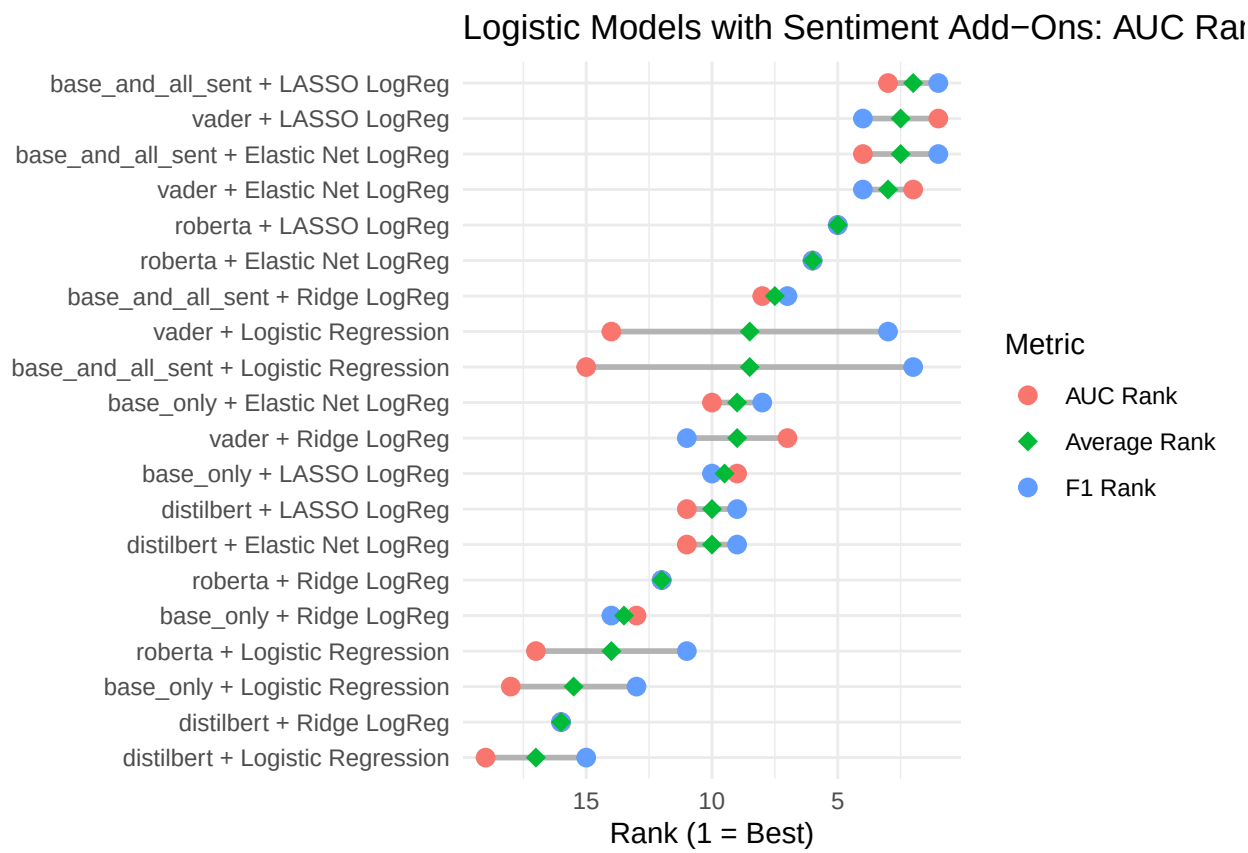
```r
      res$enet_eval$auc
    ),
    F1 = c(
      max(res$logit_eval$f1_results$F1),
      max(res$lasso_eval$f1_results$F1),
      max(res$ridge_eval$f1_results$F1),
      max(res$enet_eval$f1_results$F1)
    ),
    Best_Threshold = c(
      res$logit_eval$best_threshold,
      res$lasso_eval$best_threshold,
      res$ridge_eval$best_threshold,
      res$enet_eval$best_threshold
    ),
    Accuracy = c(
      as.numeric(res$logit_eval$confusion_matrix$overall["Accuracy"]),
      as.numeric(res$lasso_eval$confusion_matrix$overall["Accuracy"]),
      as.numeric(res$ridge_eval$confusion_matrix$overall["Accuracy"]),
      as.numeric(res$enet_eval$confusion_matrix$overall["Accuracy"])
    )
  )
}))

model_comparison %>% arrange(desc(AUC))
```

```
##             Features                Model       AUC        F1 Best_Threshold
## 1              vader        LASSO LogReg 0.9507009 0.8148148           0.65
## 2              vader  Elastic Net LogReg 0.9493942 0.8148148           0.65
## 3  base_and_all_sent        LASSO LogReg 0.9479686 0.8235294           0.35
## 4  base_and_all_sent  Elastic Net LogReg 0.9468995 0.8235294           0.35
## 5            roberta        LASSO LogReg 0.9451176 0.8125000           0.25
## 6            roberta  Elastic Net LogReg 0.9441673 0.8120301           0.20
## 7              vader        Ridge LogReg 0.9419102 0.8000000           0.65
## 8  base_and_all_sent        Ridge LogReg 0.9417914 0.8103448           0.45
## 9          base_only        LASSO LogReg 0.9417320 0.8034188           0.45
## 10         base_only  Elastic Net LogReg 0.9398907 0.8070175           0.55
## 11        distilbert        LASSO LogReg 0.9382276 0.8067227           0.35
## 12        distilbert  Elastic Net LogReg 0.9382276 0.8067227           0.40
## 13           roberta        Ridge LogReg 0.9376336 0.8000000           0.25
## 14         base_only        Ridge LogReg 0.9333571 0.7938931           0.25
## 15             vader Logistic Regression 0.9332383 0.8173913           0.40
## 16 base_and_all_sent Logistic Regression 0.9331195 0.8214286           0.50
## 17        distilbert        Ridge LogReg 0.9319316 0.7863248           0.45
## 18           roberta Logistic Regression 0.9284866 0.8000000           0.20
## 19         base_only Logistic Regression 0.9238536 0.7968750           0.25
## 20        distilbert Logistic Regression 0.9223093 0.7868852           0.35
##     Accuracy
## 1  0.8994975
## 2  0.8994975
## 3  0.8944724
## 4  0.8944724
## 5  0.8793970
## 6  0.8743719
```

```
## 7  0.8944724
## 8  0.8894472
## 9  0.8844221
## 10 0.8894472
## 11 0.8844221
## 12 0.8844221
## 13 0.8643216
## 14 0.8643216
## 15 0.8944724
## 16 0.8994975
## 17 0.8743719
## 18 0.8693467
## 19 0.8693467
## 20 0.8693467
```

5.2 Plot Logistic Sentiment Add-On Models

```r
logistic_rank_plot <- model_comparison %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
  arrange(Avg_Rank) %>%
  mutate(Model_Label = factor(Model_Label, levels = rev(Model_Label)))

ggplot(logistic_rank_plot, aes(y = Model_Label)) +
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +
  geom_point(aes(x = AUC_Rank, color = "AUC Rank"), size = 2.8) +
  geom_point(aes(x = F1_Rank, color = "F1 Rank"), size = 2.8) +
  geom_point(aes(x = Avg_Rank, color = "Average Rank"), size = 3.2, shape = 18) +
  scale_x_reverse() +
  theme_minimal() +
  labs(
    title = "Logistic Models with Sentiment Add-Ons: AUC Rank vs F1 Rank vs Average Rank",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  )
```



6. Fit Random Forest Models with Sentiment Add-Ons

```
rf_results <- list()
rf_importance <- list()

for (name in names(datasets)) {
  df_current <- datasets[[name]]

  train_df <- df_current[train_index, ]
  test_df  <- df_current[-train_index, ]

  p <- ncol(train_df) - 1

  rf_model <- train(
    flags_recommend_frend ~ .,
    data = train_df,
    method = "rf",
    trControl = trainControl(method = "cv", number = 10),
    tuneGrid = expand.grid(mtry = 1:p),
    ntree = 500,
    importance = TRUE
  )
```

```r
  bestTune <- rf_model$bestTune

  rf_model_tuned <- train(
    flags_recommend_frend ~ .,
    data = train_df,
    method = "rf",
    trControl = trainControl(
      method = "cv",
      number = 10,
      classProbs = TRUE,
      summaryFunction = twoClassSummary
    ),
    tuneGrid = bestTune,
    ntree = 500,
    metric = "ROC",
    importance = TRUE
  )

  prob_rf <- predict(rf_model_tuned, newdata = test_df, type = "prob")[, "POSITIVE"]
  eval_rf <- evaluate_model(prob_rf, test_df$flags_recommend_frend)

  rf_results[[name]] <- list(
    best_mtry = bestTune$mtry,
    rf_model = rf_model,
    rf_model_tuned = rf_model_tuned,
    eval = eval_rf
  )

  rf_importance[[paste0(name, "_rf")]] <- extract_rf_importance(
    rf_model_tuned, name, "Random Forest"
  )
}
```

6.1 Summarize Random Forest Models with Sentiment Add-Ons

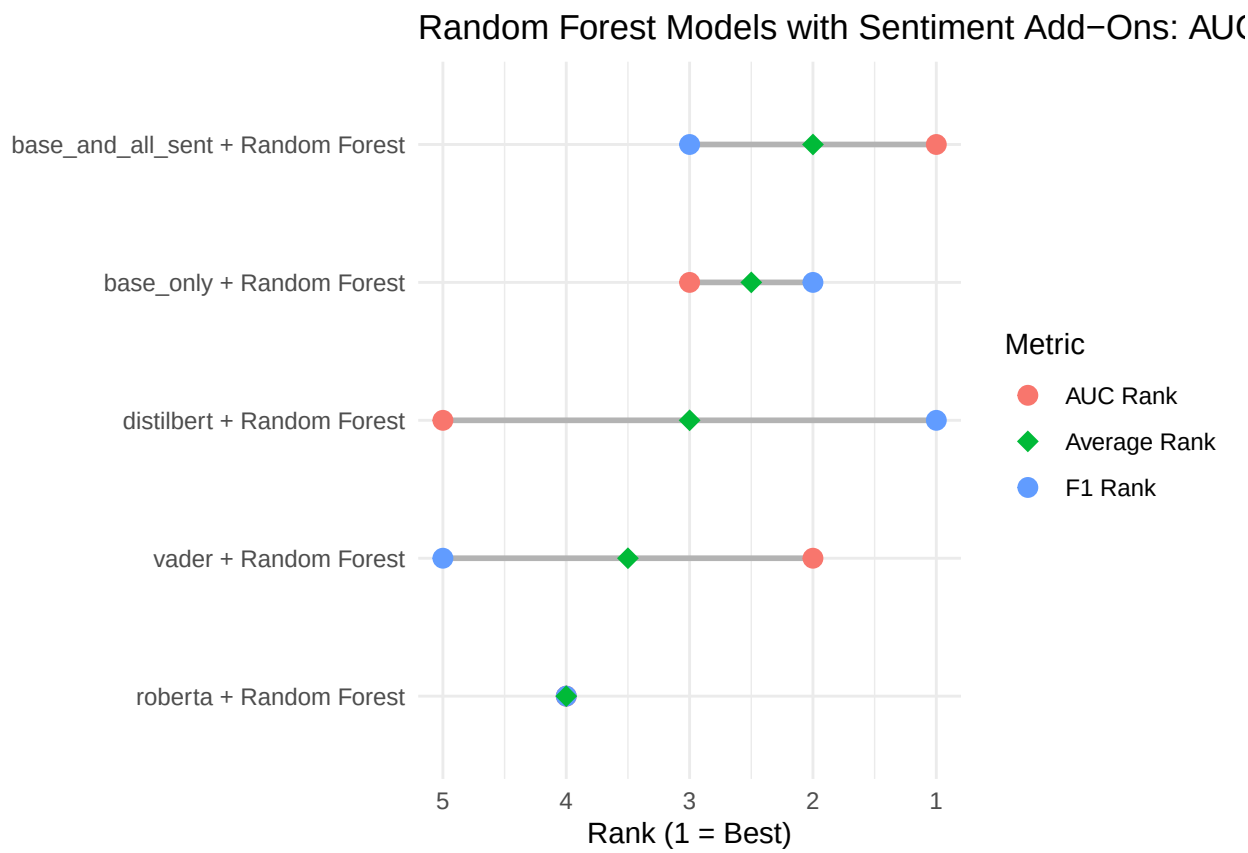
```r
rf_summary <- data.frame(
  Features = names(rf_results),
  Model = "Random Forest",
  Best_mtry = sapply(rf_results, function(x) x$best_mtry),
  AUC = sapply(rf_results, function(x) x$eval$auc),
  F1 = sapply(rf_results, function(x) max(x$eval$f1_results$F1)),
  Accuracy = sapply(rf_results, function(x) as.numeric(x$eval$confusion_matrix$overall["Accuracy"])),
  Best_Threshold = sapply(rf_results, function(x) x$eval$best_threshold),
  row.names = NULL
)

rf_summary
```

```
##            Features         Model Best_mtry       AUC        F1  Accuracy
## 1         base_only Random Forest        10 0.9350202 0.8235294 0.8944724
```

```
## 2             vader Random Forest         3 0.9388216 0.8000000 0.8844221
## 3        distilbert Random Forest        14 0.9335947 0.8275862 0.8994975
## 4           roberta Random Forest         7 0.9349608 0.8064516 0.8793970
## 5 base_and_all_sent Random Forest         9 0.9399501 0.8205128 0.8944724
##   Best_Threshold
## 1           0.40
## 2           0.45
## 3           0.50
## 4           0.35
## 5           0.45
```

6.2 Plot Random Forest Sentiment Add-On Models

```r
rf_rank_plot <- rf_summary %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
  arrange(Avg_Rank) %>%
  mutate(Model_Label = factor(Model_Label, levels = rev(Model_Label)))

ggplot(rf_rank_plot, aes(y = Model_Label)) +
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +
  geom_point(aes(x = AUC_Rank, color = "AUC Rank"), size = 3) +
  geom_point(aes(x = F1_Rank, color = "F1 Rank"), size = 3) +
  geom_point(aes(x = Avg_Rank, color = "Average Rank"), size = 3.5, shape = 18) +
  scale_x_reverse() +
  theme_minimal() +
  labs(
    title = "Random Forest Models with Sentiment Add-Ons: AUC Rank vs F1 Rank vs Average Rank",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  )
```



7. Combine All Model Results

```
logistic_df <- model_comparison %>%
  dplyr::select(Features, Model, AUC, F1, Accuracy)

rf_df <- rf_summary %>%
  dplyr::select(Features, Model, AUC, F1, Accuracy)

full_summary_df <- bind_rows(logistic_df, rf_df) %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
  arrange(Avg_Rank)

full_summary_df
```

```
##             Features                Model       AUC        F1  Accuracy
## 1  base_and_all_sent         LASSO LogReg 0.9479686 0.8235294 0.8944724
## 2  base_and_all_sent  Elastic Net LogReg 0.9468995 0.8235294 0.8944724
## 3              vader         LASSO LogReg 0.9507009 0.8148148 0.8994975
```

```
## 4              vader  Elastic Net LogReg 0.9493942 0.8148148 0.8994975
## 5            roberta        LASSO LogReg 0.9451176 0.8125000 0.8793970
## 6            roberta  Elastic Net LogReg 0.9441673 0.8120301 0.8743719
## 7  base_and_all_sent       Random Forest 0.9399501 0.8205128 0.8944724
## 8  base_and_all_sent        Ridge LogReg 0.9417914 0.8103448 0.8894472
## 9          base_only       Random Forest 0.9350202 0.8235294 0.8944724
## 10        distilbert       Random Forest 0.9335947 0.8275862 0.8994975
## 11         base_only  Elastic Net LogReg 0.9398907 0.8070175 0.8894472
## 12             vader        Ridge LogReg 0.9419102 0.8000000 0.8944724
## 13         base_only        LASSO LogReg 0.9417320 0.8034188 0.8844221
## 14 base_and_all_sent Logistic Regression 0.9331195 0.8214286 0.8994975
## 15             vader Logistic Regression 0.9332383 0.8173913 0.8944724
## 16        distilbert        LASSO LogReg 0.9382276 0.8067227 0.8844221
## 17        distilbert  Elastic Net LogReg 0.9382276 0.8067227 0.8844221
## 18             vader       Random Forest 0.9388216 0.8000000 0.8844221
## 19           roberta       Random Forest 0.9349608 0.8064516 0.8793970
## 20           roberta        Ridge LogReg 0.9376336 0.8000000 0.8643216
## 21         base_only        Ridge LogReg 0.9333571 0.7938931 0.8643216
## 22           roberta Logistic Regression 0.9284866 0.8000000 0.8693467
## 23         base_only Logistic Regression 0.9238536 0.7968750 0.8693467
## 24        distilbert        Ridge LogReg 0.9319316 0.7863248 0.8743719
## 25        distilbert Logistic Regression 0.9223093 0.7868852 0.8693467
##                                Model_Label AUC_Rank F1_Rank Avg_Rank
## 1         base_and_all_sent + LASSO LogReg        3       2      2.5
## 2   base_and_all_sent + Elastic Net LogReg        4       2      3.0
## 3                     vader + LASSO LogReg        1       6      3.5
## 4               vader + Elastic Net LogReg        2       6      4.0
## 5                   roberta + LASSO LogReg        5       7      6.0
## 6             roberta + Elastic Net LogReg        6       8      7.0
## 7        base_and_all_sent + Random Forest       10       4      7.0
## 8         base_and_all_sent + Ridge LogReg        8       9      8.5
## 9                base_only + Random Forest       15       2      8.5
## 10              distilbert + Random Forest       17       1      9.0
## 11          base_only + Elastic Net LogReg       11      10     10.5
## 12                    vader + Ridge LogReg        7      14     10.5
## 13                base_only + LASSO LogReg        9      13     11.0
## 14 base_and_all_sent + Logistic Regression       20       3     11.5
## 15             vader + Logistic Regression       19       5     12.0
## 16               distilbert + LASSO LogReg       13      11     12.0
## 17         distilbert + Elastic Net LogReg       13      11     12.0
## 18                   vader + Random Forest       12      15     13.5
## 19                 roberta + Random Forest       16      12     14.0
## 20                  roberta + Ridge LogReg       14      15     14.5
## 21                base_only + Ridge LogReg       18      17     17.5
## 22           roberta + Logistic Regression       22      14     18.0
## 23         base_only + Logistic Regression       23      16     19.5
## 24               distilbert + Ridge LogReg       21      19     20.0
## 25        distilbert + Logistic Regression       24      18     21.0
```

Interpretation:

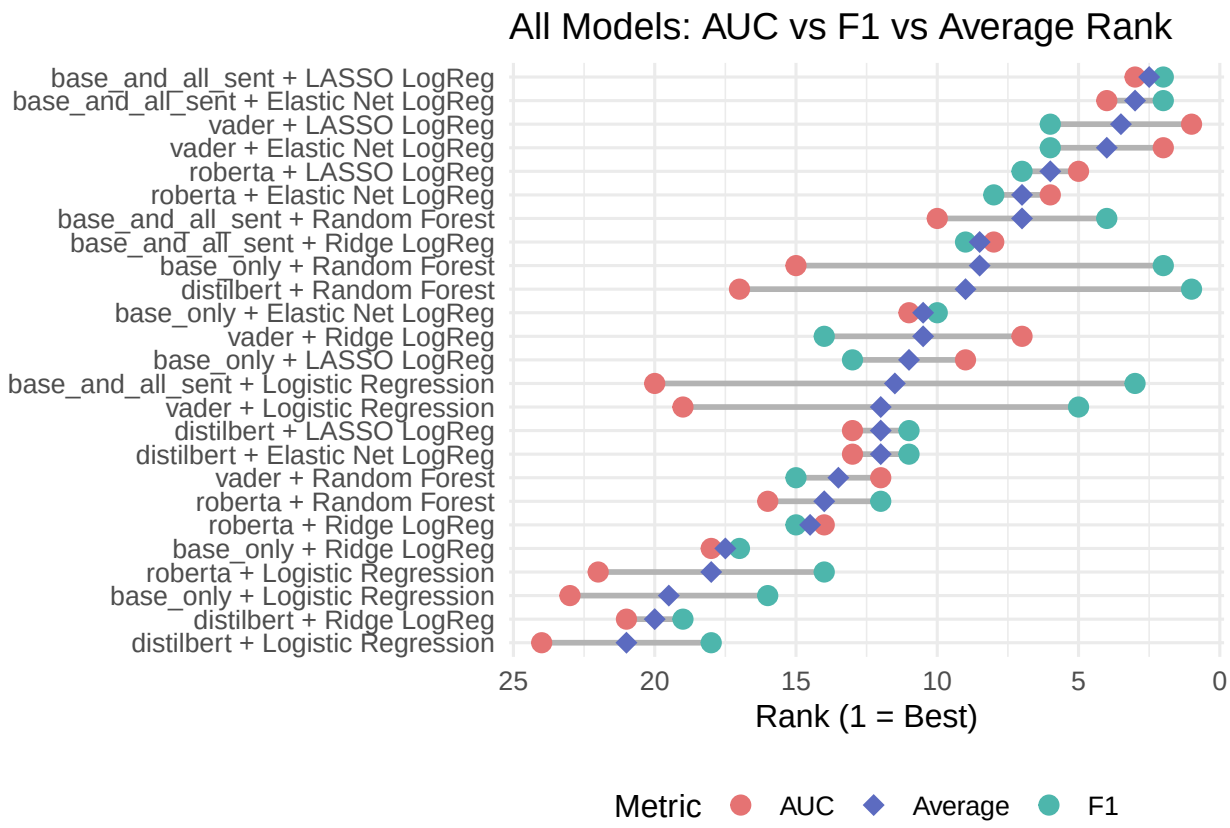
Adding sentiment features leads to noticeable improvements in model performance, particularly in AUC. Models that incorporate sentiment (especially VADER and RoBERTa) consistently rank higher than base-

only models, indicating that textual tone provides additional predictive signal.

The improvement is more pronounced for linear models than Random Forest, suggesting that sentiment features introduce strong linear separability rather than complex nonlinear patterns.

7.1 Plot Overall Model Rankings

```r
full_summary_plot <- full_summary_df %>%
  mutate(
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
  arrange(Avg_Rank) %>%
  mutate(
    Model_Label = factor(Model_Label, levels = rev(Model_Label))
  )

ggplot(full_summary_plot, aes(y = Model_Label)) +
  # Line between AUC and F1
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +

  # AUC point
  geom_point(aes(x = AUC_Rank, color = "AUC"), size = 3) +

  # F1 point
  geom_point(aes(x = F1_Rank, color = "F1"), size = 3) +

  # Avg rank point (NEW)
  geom_point(aes(x = Avg_Rank, color = "Average"), size = 3.5, shape = 18) +

  scale_x_reverse() +
  scale_color_manual(values = c(
    "AUC" = "#E57373",
    "F1" = "#4DB6AC",
    "Average" = "#5C6BC0"
  )) +

  theme_minimal(base_size = 12) +
  labs(
    title = "All Models: AUC vs F1 vs Average Rank",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  ) +
  theme(
    legend.position = "bottom"
  )
```



Interpretation:

Model rankings based on AUC and F1 are generally aligned, though some models perform better on one metric than the other. The average rank provides a balanced evaluation, identifying models that perform consistently well across both metrics.

Top-performing models combine sentiment features with regularized logistic regression, indicating that both feature richness and controlled model complexity are important for optimal performance.

8. View Top 10 Models by Average Rank

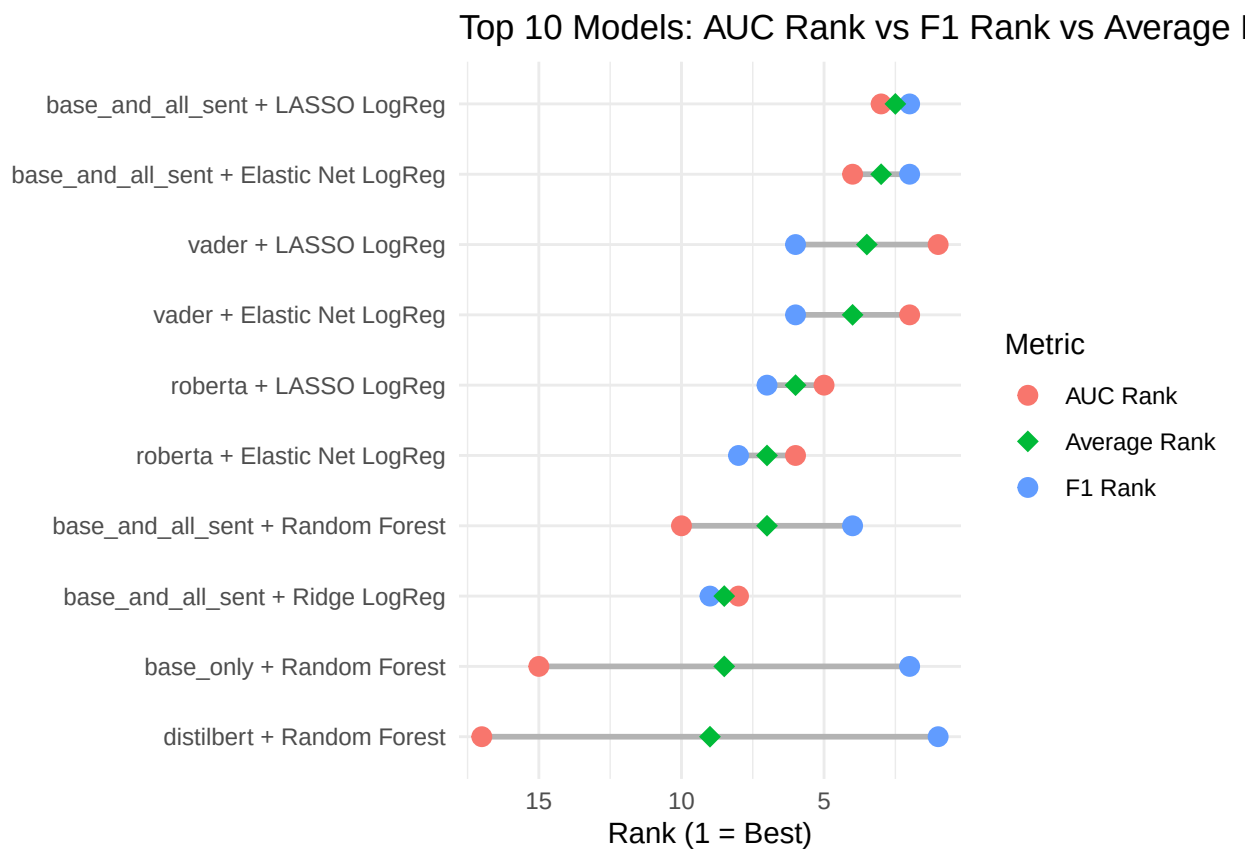
```
top_models <- full_summary_df %>%
  slice_head(n = 10)

top_models %>%
  dplyr::select(Model_Label, AUC, F1, Accuracy, AUC_Rank, F1_Rank, Avg_Rank)
```

```
##                                Model_Label       AUC        F1  Accuracy
## 1        base_and_all_sent + LASSO LogReg 0.9479686 0.8235294 0.8944724
## 2  base_and_all_sent + Elastic Net LogReg 0.9468995 0.8235294 0.8944724
## 3                     vader + LASSO LogReg 0.9507009 0.8148148 0.8994975
## 4              vader + Elastic Net LogReg 0.9493942 0.8148148 0.8994975
```

```
## 5                  roberta + LASSO LogReg 0.9451176 0.8125000 0.8793970
## 6            roberta + Elastic Net LogReg 0.9441673 0.8120301 0.8743719
## 7       base_and_all_sent + Random Forest 0.9399501 0.8205128 0.8944724
## 8        base_and_all_sent + Ridge LogReg 0.9417914 0.8103448 0.8894472
## 9               base_only + Random Forest 0.9350202 0.8235294 0.8944724
## 10             distilbert + Random Forest 0.9335947 0.8275862 0.8994975
##    AUC_Rank F1_Rank Avg_Rank
## 1         3       2      2.5
## 2         4       2      3.0
## 3         1       6      3.5
## 4         2       6      4.0
## 5         5       7      6.0
## 6         6       8      7.0
## 7        10       4      7.0
## 8         8       9      8.5
## 9        15       2      8.5
## 10       17       1      9.0
```

8.1 Plot Top 10 Models by Average Rank

```r
top_models_plot <- top_models %>%
  arrange(Avg_Rank) %>%
  mutate(Model_Label = factor(Model_Label, levels = rev(Model_Label)))

ggplot(top_models_plot, aes(y = Model_Label)) +
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +
  geom_point(aes(x = AUC_Rank, color = "AUC Rank"), size = 3) +
  geom_point(aes(x = F1_Rank, color = "F1 Rank"), size = 3) +
  geom_point(aes(x = Avg_Rank, color = "Average Rank"), size = 3.5, shape = 18) +
  scale_x_reverse() +
  theme_minimal() +
  labs(
    title = "Top 10 Models: AUC Rank vs F1 Rank vs Average Rank",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  )
```



9. Combine All Feature Importance Results

```
importance_all <- bind_rows(
  bind_rows(logit_importance),
  bind_rows(rf_importance)
) %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    term_clean = clean_feature_names(term)
  )

importance_all
```

```
## # A tibble: 250 x 8
##    Features  Model    term  estimate Importance Direction Model_Label term_clean
##    <chr>     <chr>    <chr>    <dbl>      <dbl> <chr>     <chr>       <chr>
##  1 base_only Logisti~ sect~    11.6       11.6  Positive  base_only ~ sectorLeg~
##  2 base_only Logisti~ sect~    11.5       11.5  Positive  base_only ~ sectorEne~
##  3 base_only Logisti~ flag~    11.4       11.4  Positive  base_only ~ ceo appro~
##  4 base_only Logisti~ flag~    11.2       11.2  Positive  base_only ~ ceo appro~
##  5 base_only Logisti~ sect~    11.2       11.2  Positive  base_only ~ sectorRea~
##  6 base_only Logisti~ flag~    10.2       10.2  Positive  base_only ~ ceo appro~
##  7 base_only Logisti~ sect~    10.1       10.1  Positive  base_only ~ sectorCon~
```

```
##  8 base_only Logisti~ sect~     9.79       9.79 Positive  base_only ~ sectorEdu~
##  9 base_only Logisti~ sect~     9.61       9.61 Positive  base_only ~ sectorTra~
## 10 base_only Logisti~ sect~     9.54       9.54 Positive  base_only ~ sectorMed~
## # i 240 more rows
```

9.1 View Top 10 Features for the Best Overall Models

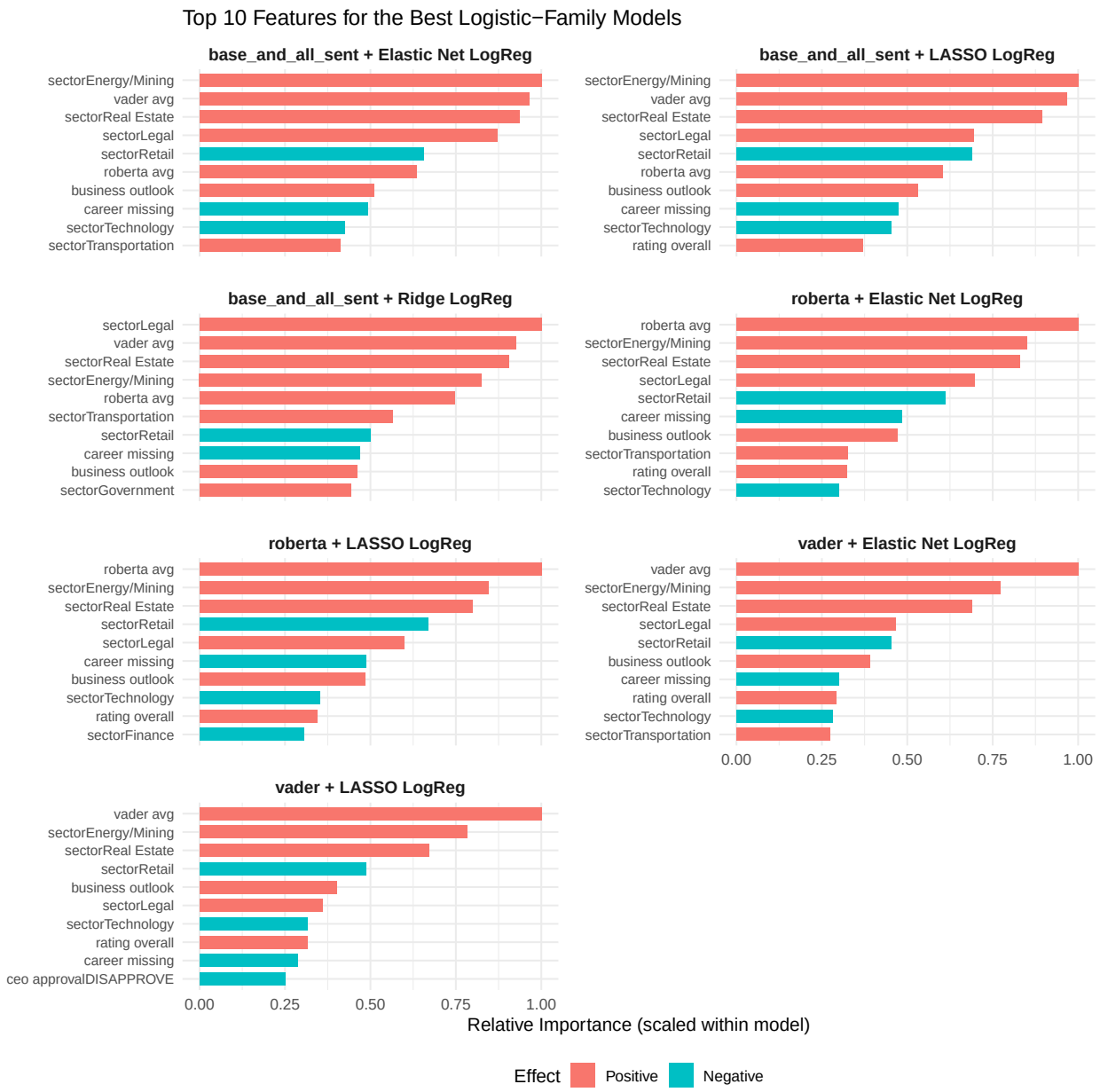
```
top_model_features_df <- importance_all %>%
  semi_join(top_models, by = "Model_Label") %>%
  group_by(Model_Label) %>%
  slice_max(order_by = Importance, n = 10, with_ties = FALSE) %>%
  mutate(Importance_scaled = Importance / max(Importance)) %>%
  ungroup() %>%
  arrange(Model_Label, desc(Importance_scaled))

top_model_features_df
```

```
## # A tibble: 100 x 9
##    Features     Model term  estimate Importance Direction Model_Label term_clean
##    <chr>        <chr> <chr>    <dbl>      <dbl> <chr>     <chr>       <chr>
##  1 base_and_al~ Elas~ sect~    1.85       1.85  Positive  base_and_a~ sectorEne~
##  2 base_and_al~ Elas~ sent~    1.78       1.78  Positive  base_and_a~ vader avg
##  3 base_and_al~ Elas~ sect~    1.73       1.73  Positive  base_and_a~ sectorRea~
##  4 base_and_al~ Elas~ sect~    1.61       1.61  Positive  base_and_a~ sectorLeg~
##  5 base_and_al~ Elas~ sect~   -1.21       1.21  Negative  base_and_a~ sectorRet~
##  6 base_and_al~ Elas~ sent~    1.18       1.18  Positive  base_and_a~ roberta a~
##  7 base_and_al~ Elas~ flag~    0.942      0.942 Positive  base_and_a~ business ~
##  8 base_and_al~ Elas~ care~   -0.912      0.912 Negative  base_and_a~ career mi~
##  9 base_and_al~ Elas~ sect~   -0.783      0.783 Negative  base_and_a~ sectorTec~
## 10 base_and_al~ Elas~ sect~    0.759      0.759 Positive  base_and_a~ sectorTra~
## # i 90 more rows
## # i 1 more variable: Importance_scaled <dbl>
```

9.2 Plot Top 10 Features for the Best Logistic-Family Models

```
top_logit_models <- top_models %>%
  filter(Model != "Random Forest")

top_logit_features_df <- importance_all %>%
  semi_join(top_logit_models, by = "Model_Label") %>%
  filter(Model != "Random Forest") %>%
  group_by(Model_Label) %>%
  slice_max(order_by = Importance, n = 10, with_ties = FALSE) %>%
  mutate(
    Importance_scaled = Importance / max(Importance)
  ) %>%
  ungroup()
```

```r
top_logit_features_plot <- top_logit_features_df %>%
  mutate(
    term_plot = tidytext::reorder_within(term_clean, Importance_scaled, Model_Label),
    Direction = factor(Direction, levels = c("Positive", "Negative"))
  )

ggplot(top_logit_features_plot, aes(x = term_plot, y = Importance_scaled, fill = Direction)) +
  geom_col(width = 0.7) +
  coord_flip() +
  tidytext::scale_x_reordered() +
  facet_wrap(~ Model_Label, ncol = 2, scales = "free_y") +
  theme_minimal(base_size = 12) +
  labs(
    title = "Top 10 Features for the Best Logistic-Family Models",
    x = NULL,
    y = "Relative Importance (scaled within model)",
    fill = "Effect"
  ) +
  theme(
    axis.text.y = element_text(size = 9),
    strip.text = element_text(size = 11, face = "bold"),
    legend.position = "bottom",
    panel.spacing = unit(1.2, "lines")
  )
```



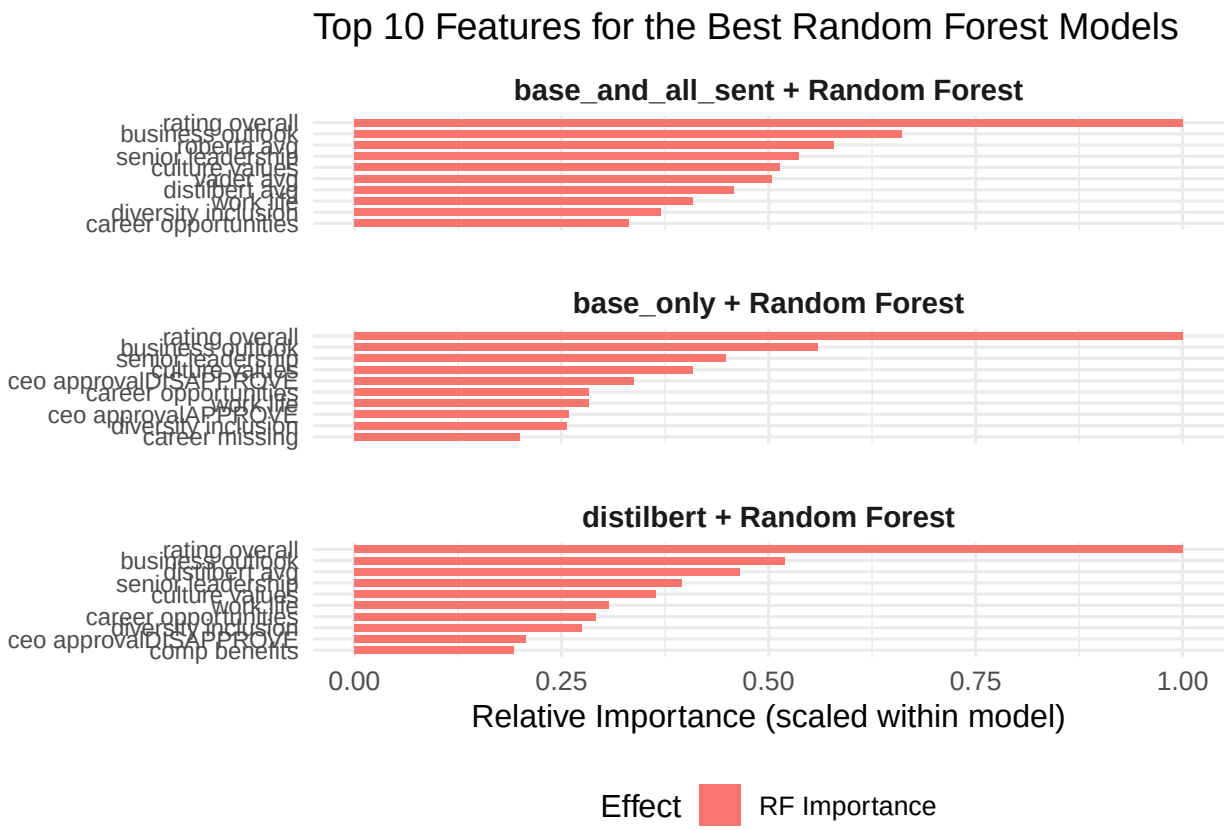
Interpretation

Sentiment features—particularly VADER average sentiment—emerge as the most influential predictors in the best-performing models. This indicates that overall tone of the review is a strong signal for recommendation behavior.

Additionally, sentiment variability (e.g., standard deviation) appears in some models, suggesting that inconsistency across different review sections also carries predictive value. Traditional structured features remain relevant but are often secondary to sentiment in importance rankings.

9.3 Top 10 Features for Best Random Forest Model

```r
top_rf_models <- top_models %>%
  filter(Model == "Random Forest")

if (nrow(top_rf_models) == 0) {
  cat("No Random Forest models appear in the top models table.\n")
} else {
  top_rf_features_df <- importance_all %>%
    semi_join(top_rf_models, by = "Model_Label") %>%
    filter(Model == "Random Forest") %>%
    group_by(Model_Label) %>%
    slice_max(order_by = Importance, n = 10, with_ties = FALSE) %>%
    mutate(
      Importance_scaled = Importance / max(Importance, na.rm = TRUE)
    ) %>%
    ungroup()

  if (nrow(top_rf_features_df) == 0) {
    cat("No Random Forest feature importance rows available for plotting.\n")
  } else {
    top_rf_features_plot <- top_rf_features_df %>%
      mutate(
        term_plot = tidytext::reorder_within(term_clean, Importance_scaled, Model_Label),
        Direction = factor(Direction, levels = c("RF Importance"))
      )

    ggplot(top_rf_features_plot, aes(x = term_plot, y = Importance_scaled, fill = Direction)) +
      geom_col(width = 0.7) +
      coord_flip() +
      tidytext::scale_x_reordered() +
      facet_wrap(~ Model_Label, ncol = 1, scales = "free_y") +
      theme_minimal(base_size = 12) +
      labs(
        title = "Top 10 Features for the Best Random Forest Models",
        x = NULL,
        y = "Relative Importance (scaled within model)",
        fill = "Effect"
      ) +
      theme(
        axis.text.y = element_text(size = 9),
        strip.text = element_text(size = 11, face = "bold"),
        legend.position = "bottom",
        panel.spacing = unit(1.2, "lines")
      )
  }
}
```



Final Logistic vs RF Interpretation

Logistic regression models provide clearer and more interpretable feature effects, while Random Forest captures broader patterns but with less interpretability. The fact that logistic-based models perform competitively—or better—suggests that the relationship between features and the target variable is largely linear and well-structured.

This reinforces the effectiveness of feature engineering (especially sentiment) over increasing model complexity (ie. regularization).